# Bangladesh University of Engineering and Technology

**CSE 208: Data Structure and Algorithm II Sessional**

**Student ID: 2205157**

**MD ABU RUSSEL**

# Hashing Performance Analysis Report

- Detailed description of the hash functions used (Hash1 and Hash2) and their integration.
- Constants utilized in the experimental setup.
- Detailed performance tables for load factors ranging from 0.4 to 0.9.
- Analysis of the impact of load factors on overall hashing performance.

## 1. Hash Function Descriptions

Two specific hash functions, 'hash1' and 'hash2', were implemented within the 'HashTable' class and employed as primary hash functions to evaluate their influence on hash table behavior. The design of these functions is crucial as it directly affects the distribution of keys and, consequently, the number of and overall performance.

### 1.1 Hash1 Function: Polynomial Rolling Hash

The Hash1 function implements a widely used polynomial rolling hash algorithm, particularly effective for string hashing. It iterates through each character c of the input string word and progressively computes a hash value using the following recurrence:

$$\text{hash} = \left(\text{hash} \times 31\ \text{ASCII(c)}\right)\ \bmod \text{N}$$

alternatively, it can be expressed as:

$$\boldsymbol{hash(S) = (\sum_{i=0}^{n-1} ASCII\ (s_i).p^i)\ \ mod\ N}$$

**Where:**

- 31 is a commonly chosen small prime multiplier known to provide good hash value dispersion,
- ASCII(c) is the numeric value of the character,
- N is the size of the hash table.

The modulo operation ensures that the resulting hash value stays within the bounds of the table size and avoids overflow.

**Reasons to Use:**

- Uniform Distribution:
  This hashing method is recognized for producing a well-distributed range of hash values, especially when the table size N is a prime number, minimizing clustering and improving performance in hash-based structures.
- Computational Efficiency:
  The algorithm uses only simple arithmetic operations (multiplication, addition, and modulo) per character, making it highly efficient for large-scale string processing.
- Low Collision Probability:
  The use of a prime multiplier (like 31) helps spread similar strings apart in hash space, reducing the likelihood of collisions and enhancing overall reliability.

**Integration into System:** For the "Hash1 Function" columns in the performance tables, this 'hash1' function is used as the primary hash function to determine the initial bucket index for all three hashing methods: Separate Chaining, Linear Probing, and Double Hashing.

### 1.2 Hash2 Function: DJB2 String Hash

This function implements the **DJB2 hash algorithm**, a popular and widely used string hashing method developed by **Daniel J. Bernstein**. The procedure is as follows:

1. Initializes the hash to a **seed value** of 5381, a prime number chosen for its empirical effectiveness.
2. Iterates through each character c of the input string word.
3. For each character, it performs:

$$\boldsymbol{hash = \left((hash \ll 5) + hash\right) + c = hash \times 33 + c}$$

   This multiplication by 33 (via bit shifting and addition) allows fast and efficient hashing.

4. The final result is taken modulo N-1 and 1 is added:

$$\boldsymbol{return\ value = \left(hash\ mod (N - 1)\right) + 1}$$

   This ensures the returned hash is in the range, making it suitable for step sizes in **double hashing**.

**Reasons to Use:**

- **Excellent Distribution**:
  DJB2 offers a robust hash spread with **low collision rates**, even for similar string inputs.
- **High Efficiency**:
  It uses only **bitwise shifts and additions**, which are extremely fast on modern processors.

- **Widely Adopted**:
  This hash function has seen adoption in many **open-source projects** due to its simplicity and strong empirical performance.
- **Ideal for Double Hashing**:
  The use of guarantees a **non-zero step size**, which is essential when used as a secondary hash function in **open addressing schemes**.

**Integration into System:** For the "Hash2 Function" columns in the performance tables, this 'hash2' function is used as the primary hash function to determine the initial bucket index for all three hashing methods: Separate Chaining, Linear Probing, and Double Hashing.

## 2. Functional Constants

**Size of Hash Table (N):** 5003

**Small Prime Number (S)**: 5

**Load Factors Evaluated:** 0.4, 0.5, 0.6, 0.7, 0.8, 0.9

**Time Unit:** All "Time" values are presented in ns (nano second).

## 3. Performance Tables:

For Load Factor = 0.4

| Method | Hash Function 1 | | | | | Hash Function 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | # of collisions | Before Deletion | | After Deletion | | # of collisions | Before Deletion | | After Deletion | |
| | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes |
| Linear Probing | 694 | 270.33 | 1.2875 | 461.46 | 2.140625 | 674 | 213.41 | 1.203125 | 264.96 | 2.29375 |
| Separate Chaining | 422 | 326.3 | N/A | 290.54 | N/A | 424 | 212.22 | N/A | 199.6 | N/A |
| Double Hashing | 616 | 383.43 | 1.203125 | 840.77 | 1.89375 | 656 | 325.27 | 1.2125 | 419.52 | 2.04375 |

For Load Factor = 0.5

| Method | Hash Function 1 | | | | | Hash Function 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | # of collisions | Before Deletion | | After Deletion | | # of collisions | Before Deletion | | After Deletion | |
| | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes |
| Separate Chaining | 620 | 215.97 | N/A | 147.37 | N/A | 593 | 203.85 | N/A | 129.98 | N/A |
| Linear Probing | 1309 | 206.69 | 1.3225 | 213.23 | 2.9775 | 1136 | 182.9 | 1.23 | 228.31 | 2.675 |
| Double Hashing | 1030 | 300.32 | 1.3325 | 283.6 | 3.025 | 921 | 274.03 | 1.2175 | 268.18 | 2.225 |

For Load Factor = 0.6

| Method | Hash Function 1 | | | | | Hash Function 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | # of collisions | Before Deletion | | After Deletion | | # of collisions | Before Deletion | | After Deletion | |
| | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes |
| Separate Chaining | 764 | 196.88 | N/A | 130.37 | N/A | 799 | 169.82 | N/A | 115.43 | N/A |
| Linear Probing | 1714 | 181.43 | 1.4375 | 238.38 | 4.55 | 1803 | 168.66 | 1.454167 | 175.94 | 2.741667 |
| Double Hashing | 1338 | 262.4 | 1.39375 | 248.44 | 3.0875 | 1345 | 244.28 | 1.347917 | 225.59 | 2.302083 |

For Load Factor = 0.7

| Method | Hash Function 1 | | | | | Hash Function 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | # of collisions | Before Deletion | | After Deletion | | # of collisions | Before Deletion | | After Deletion | |
| | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes |
| Separate Chaining | 980 | 168.67 | N/A | 124.4 | N/A | 1022 | 136.3 | N/A | 99.7 | N/A |
| Linear Probing | 2482 | 168.44 | 1.448214 | 273.14 | 6.414286 | 2690 | 159.62 | 1.548214 | 195.6 | 3.980357 |
| Double Hashing | 1867 | 238.47 | 1.351786 | 250.22 | 3.757143 | 2007 | 225.56 | 1.391071 | 209.6 | 2.517857 |

For Load Factor = 0.8

| Method | Hash Function 1 | | | | | Hash Function 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | # of collisions | Before Deletion | | After Deletion | | # of collisions | Before Deletion | | After Deletion | |
| | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes |
| Separate Chaining | 1206 | 134.52 | N/A | 112.36 | N/A | 1181 | 146.72 | N/A | 113.03 | N/A |
| Linear Probing | 3734 | 152.83 | 1.604688 | 227.12 | 5.585938 | 3386 | 154.87 | 1.7125 | 236.58 | 4.178125 |
| Double Hashing | 2479 | 208.9 | 1.407812 | 208.32 | 3.471875 | 2404 | 213.03 | 1.485937 | 221.68 | 2.657812 |

For Load Factor = 0.9

| Method | Hash Function 1 | | | | | Hash Function 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | # of collisions | Before Deletion | | After Deletion | | # of collisions | Before Deletion | | After Deletion | |
| | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes | | Avg S.Time | Avg Probes | Avg S.Time | Avg Probes |
| Separate Chaining | 1368 | 169.7 | N/A | 110.48 | N/A | 1392 | 152.02 | N/A | 93.66 | N/A |
| Linear Probing | 4372 | 169.63 | 1.791667 | 316.68 | 8.858333 | 4776 | 159.68 | 1.806944 | 215.08 | 5.133333 |
| Double Hashing | 3154 | 217.36 | 1.5625 | 238.85 | 4.534722 | 3160 | 217.07 | 1.477778 | 200.23 | 2.844444 |

## 4. Impact of Load Factors on Hashing Performance

The load factor $\alpha$, defined as the ratio of the number of elements to the hash table's size, is a critical parameter influencing the efficiency of hash table operations. As the load factor increases, the density of stored elements rises, leading to more frequent collisions and, consequently, degraded performance across various hashing methods.

### 4.1. Collision Frequency (Based on Observed Performance)

As the load factor increases, the number of collisions during insertions rises across all methods — an expected outcome since more keys compete for a limited number of slots. This is indirectly reflected in the growing search times and probe counts. At higher load factors (e.g., $\alpha \geq 0.7$), open addressing methods (Linear Probing and Double Hashing) experience a sharp increase in collisions, while Separate Chaining handles higher densities more gracefully due to its bucket-based structure.

### 4.2. Average Search Time and Probes

**Separate Chaining with Balanced BST:**

This method exhibits remarkable stability even at high load factors. Search times remain low, and the number of probes is effectively irrelevant, since within a chain, operations take logarithmic time due to the use of balanced BSTs. This makes Separate Chaining resilient to increasing α, as the chains grow only slightly deeper while remaining efficient to traverse. The "N/A" for probes reflects that probing does not occur in the same way as in open addressing.

**Linear Probing with Step Adjustment:**

Linear Probing suffers significantly as α increases. Average probe counts and search times rise steeply, particularly beyond $\alpha = 0.7$. This behavior is caused by primary clustering, where consecutive slots fill up and collisions lead to longer sequences of occupied cells. Although Step Adjustment and Hash2 sometimes slightly mitigate this effect, the performance degradation at high load factors remains substantial.

**Double Hashing:**

Double Hashing performs better than Linear Probing under higher load factors because it uses a secondary hash function to calculate step sizes, which reduces clustering. The increase in probes and search time is more gradual compared to Linear Probing, and it remains relatively efficient even when the table is more than 70–80% full. Nevertheless, at very high load factors ($\alpha \geq 0.9$), performance still degrades compared to lower load factors, though less severely than Linear Probing.

### 4.3. Impact of Hash Functions

Comparing the two hash functions — Hash1 (Polynomial Rolling Hash) and Hash2 (DJB2) — reveals some subtle differences:

- For Separate Chaining, the choice of hash function has minimal effect because collisions are efficiently managed in chains.

- For open addressing, the choice can impact distribution and, consequently, clustering.

Hash1 tends to produce slightly better or comparable results for Double Hashing across most load factors, while Hash2 occasionally shows better average search times for Linear Probing at very high load factors (e.g., $\alpha \approx 0.9$).

These variations indicate that the interaction between the hash function and the collision resolution method influences performance, especially for open addressing.

### 4.4. Effect of Deletion

The "After Deletion" results provide insight into how each method handles removals:

- Separate Chaining: Deletions have negligible impact because nodes are simply removed from their respective BSTs, maintaining fast operations.

- Linear Probing: After deletions, probe counts and search times often increase, likely due to the presence of tombstones (markers left in deleted slots), which elongate probe sequences during subsequent searches.

- Double Hashing: Similar to Linear Probing, deletions result in a modest increase in search cost, though the effect is generally less pronounced than in Linear Probing.

In summary, Separate Chaining with BSTs demonstrates strong resilience to high load factors and deletions, maintaining stable performance. Double Hashing offers a good compromise for open addressing, handling higher densities better than Linear Probing. Linear Probing, while simple, is most sensitive to high load factors and deletions due to clustering and probe sequence growth.